

Adaptation Fidelity of SPEC CPU[®]2026

Doa'a Al-Otoom, Mahesh Madhav
Ampere Computing, Portland, OR

Abstract—Standardized benchmarks are often criticized for not being “real workloads,” but this critique is rarely backed by data. This paper provides the first systematic, quantitative analysis of the “fidelity gap” between the SPEC CPU@2026 suite and its original, upstream open-source counterparts. We compile both the SPEC benchmarks and their upstream applications and execute them with official input workloads under two scenarios: a single-copy latency run and a 192-copy throughput run.

Our findings show that most benchmarks exhibit high fidelity in single-copy runs, while a few outliers reveal the impact of SPEC’s adaptation process. The multi-copy results further highlight the necessity of this adaptation: several benchmarks become significantly more efficient than their upstream versions under heavy load, underscoring the importance of I/O reduction. This work offers data-driven validation of SPEC’s methodology, showing that the fidelity gap is not a flaw but a quantifiable consequence of enforcing portability, determinism, and CPU-centric measurement.

I. INTRODUCTION

Standardized benchmarks are the bedrock of quantitative research in computer architecture, yet their very nature creates a philosophical dilemma. To be effective, they must be derived from “real workloads,” a principle the SPEC CPU committee embraces by founding each benchmark on a real-world application and preserving its essential performance characteristics [19]. To be scientific, however, they must be transformed into deterministic, portable, and reproducible instruments [11]. This transformation, which removes randomness, I/O, and platform-specific code, is essential for fair CPU comparisons but raises an important question: how much realism is sacrificed for the sake of rigor?

This question is often at the heart of critiques that dismiss benchmarks as “synthetic” [4], [17], [20]. While these concerns are valid, they are rarely substantiated with quantitative evidence; the fidelity gap between an application and its benchmark derivative has remained a subject of speculation rather than systematic study. While there have been numerous studies on how representative SPEC CPU is of industry workloads [3], [12] or other benchmarks [9], none have focused on the loss of fidelity from the adaptation process itself.

This paper provides the first comprehensive, empirical investigation into this fidelity gap for SPEC CPU@2026. For each of the 26 benchmarks in SPECrate, we compile its original upstream open-source counterpart and measure its performance against the official benchmark version using identical input workloads, precisely quantifying the delta introduced by adaptation. By analyzing these deltas, we identify which benchmarks serve as high-fidelity proxies for their in-field counterparts and which have been more significantly

perturbed; for the latter, we attribute the divergence to specific modifications such as the removal of intrinsics or I/O operations.

Ultimately, this paper offers a more nuanced, data-driven understanding of where SPEC CPU 2026 accurately reflects real-world performance and where its role as a standardized instrument causes it to diverge. The following sections detail our methodology, present our findings, and discuss the implications for benchmark users and developers alike.

Sentinel at the Gate. The SPEC CPU documentation clearly states that benchmarks adapted from applications may not look and feel like the original application:

SPEC’s Warning [15]: The individual benchmarks in this suite may be similar, but are NOT identical to benchmarks or programs with similar names available from sources other than SPEC. SPEC has invested significant effort to improve portability and minimize hardware dependencies, ... so the application programs in this distribution may perform differently from commercial or open-source versions of the same application. Therefore, it is not valid to compare SPEC CPU 2026 results with anything other than other SPEC CPU 2026 results.

This adaptation process is even laid out in detail in the SPEC CPU 2026 announcement paper [11]. Nevertheless, we throw caution to the wind and roll up our sleeves...

II. METHODOLOGY

To empirically quantify the fidelity gap between the originally snapshotted open-source applications and their adapted benchmark counterparts, we designed a controlled experiment centered on building two sets of binaries from a common source baseline, executing them in an identical environment, and verifying the results.

A. Build and Execution

First, we compile the “Upstream Application” binaries from the “redistributable source” package provided by SPEC for each benchmark, which is the code snapshot that SPEC started with. Second, we compile the official SPEC binaries, the “Adapted Benchmark,” directly using the SPEC CPU harness after the full adaptation and code-hardening process (removal of I/O, non-determinism, and platform-specific code). Both are built with `gcc-15.2` and `-O3`, based on the example `gcc` configuration offered by SPEC.

To isolate code changes as the only variable, we modify each benchmark’s `speccmds.cmd` file to replace the call to the adapted binary with a call to our compiled upstream binary, then execute it via `specinvoke` with the same input workloads, command-line arguments, and environment variables as

the official run. A few exceptions required deviation from this process; these are noted in [section IV](#).

All experiments were conducted on AArch64. While SPEC benchmarks are designed for portability, many upstream counterparts are developed on x86, so porting to a non-primary architecture serves as a useful stress test and can amplify fidelity gaps related to system interactions, since system call overhead can be higher on AArch64 [5].

Performance was measured under two load scenarios, both plotted as S-curves sorted by speedup [Figure 1](#), [Figure 2](#) and summarized in [Table II](#). The single-copy run uses the official refrate workload size (the SPECrate submission standard) and measures elapsed runtime for a single invocation of each binary, capturing performance under minimal system load. The multi-copy run scales this to a 192-copy SPECrate execution on a 192-core processor, amplifying system-level effects such as residual I/O or shared-resource contention that a single-copy run may hide.

B. Verification

A core tenet of the SPEC methodology is not just to measure how fast a workload runs, but how fast it runs correctly. To uphold this principle and ensure that the upstream applications performed an equivalent quantum of work, we integrated the official SPEC verification process into our experiment. After each run of the upstream application binary, we execute the `specinvoke compare.cmd` command provided by the harness. This command validates the application’s output against the golden reference output files, checking for correctness within the specified tolerances for each benchmark. In the vast majority of cases the upstream output matched the reference, confirming equivalent work; discrepancies were analyzed manually (e.g., in 748.flightdm, SPEC removed redundant configuration dumps and verbose step logging; in 729.abc, SPEC removed a build timestamp printout). This step ensures measured deltas reflect execution efficiency rather than differences in work performed.

C. Platform

All measurements were conducted on an AArch64 platform based on the AmpereOne SoC [1], configured as shown in [Table I](#). The results shown in this paper are reproducible on this platform; any other platform will behave differently and may even report opposite results compared to what is observed below. We offer root causes for the performance discrepancies we observed, summarized in [Table II](#) with outliers detailed in [section IV](#).

III. RESULTS

Our initial plan was to test all 26 benchmarks; three were excluded. 772.marian has been modified so heavily that it bears little resemblance to the original workload, which doesn’t even build on AArch64. For 749.fotonik3d and 782.lbm, no public reference workloads or redistributable sources exist, as these codes were contributed directly to SPEC by their authors. We therefore focus on the remaining 23 benchmarks.

TABLE I: Experimental System Configuration

Component	Configuration
Processor	AmpereOne A192-32X [1]
Cores	192 Ampere Cores (aarch64, ARMv8.6)
Frequency	3.2 GHz
Memory	1.0 TB DDR5-5600
L1 Cache	16 KB code + 64 KB data
L2 Cache	2 MB
L3 Cache	64 MB SLC
Compiler and Flags	GCC 15.2 -O3
Operating Environment	Ubuntu 24.04.4 LTS Linux kernel: 6.8.0-110-generic-64k
NUMA	1 Node per socket (Monolithic)

TABLE II: Performance comparison table. This is a summary of the data in [Figures 1](#) and [2](#). Green means SPEC is faster, red means SPEC is slower.

benchmark	1-copy	192-copy	causes of difference
706.stockfish	0.615	0.755	intrinsic removal
707.ntest	1.032	1.711	i/o efficiency
708.sqlite	1.442	1.309	changed fp types
709.cactus	1.013	1.015	
710.omnetpp	1.078	1.047	threading overhead
714.cpython	0.944	0.964	no inlines
721.gcc	1.140	1.031	system time reduction
722.palm	1.033	1.001	
723.llvm	1.076	1.361	i/o write reduction
727.cppcheck	0.978	1.001	
729.abc	0.985	0.962	
731.astcenc	0.642	0.643	intrinsic removal
734.vpr	0.974	0.921	no inlines, stable sort
735.gem5	1.113	1.378	i/o write reduction
736.ocio	1.001	0.998	
737.gmsh	1.029	1.006	
748.flightdm	1.012	1.017	i/o efficiency
749.fotonik3d	[N/A]	[N/A]	no upstream code
750.sealcrypto	1.042	1.034	
753.ns3	1.018	1.037	self-timing removal
765.roms	0.974	1.000	
766.femflow	1.176	1.077	threading overhead
767.nest	1.179	1.052	RNG, thread overheads
772.marian	[N/A]	[N/A]	does not build on arm
777.zstd	0.943	0.972	__builtin removals
782.lbm	[N/A]	[N/A]	no upstream code

Execution time was measured for two build configurations within the SPEC harness: the official benchmark binary, and the original sources pre-adaptation. Measurements were collected in single-copy mode (single-threaded performance) and 192-copy mode (throughput under full system load), plotted as S-curves sorted by speedup in [Figure 1](#) and [Figure 2](#), and tabulated in [Table II](#).

Single-copy performance speedup
SPEC CPU®2026 benchmarks
evaluated against upstream baseline

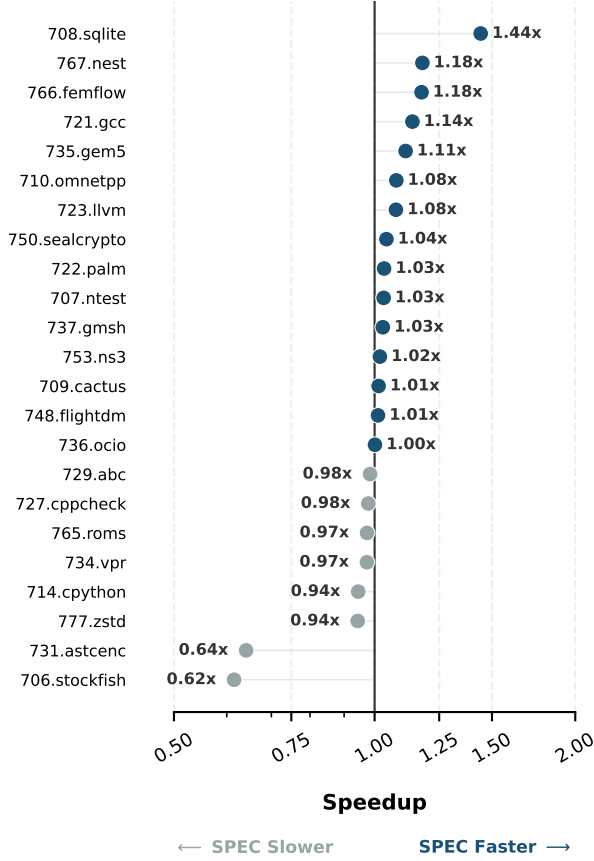


Fig. 1: Single-threaded performance comparison with a single copy running on AmpereOne. Values greater than 1.0 indicate speedup relative to the original application.

A. Single-Copy Results

Figure 1 shows the ratio of SPEC to upstream performance across all 23 benchmarks, falling into three groups. The first group ran notably faster under SPEC (ratios $1.08\times$ – $1.44\times$), primarily due to the deliberate removal of I/O and other system-level bottlenecks. The second group clusters around $1.00\times$, within run-to-run variation. The third group sits well below $1.00\times$, indicating benchmarks where the upstream build outperformed the SPEC binary, which are 706.stockfish, and 731.astcenc; their performance deltas are attributable to the necessary removal of non-portable, hand-tuned assembly or ISA intrinsics. SPEC also suppresses threading/mutex overhead in single-threaded benchmarks, masks non-standard extensions like “always inline” in favor of ISO-standard code, and removes randomness that would otherwise contend for the system RNG. In all outlier scenarios, the divergence is a direct consequence of SPEC’s methodology, where the benchmark is modified to offer portable code, reproducibility of results, and to keep the spotlight on the computational workload.

Multi-copy performance speedup
SPEC CPU®2026 benchmarks
evaluated against upstream baseline

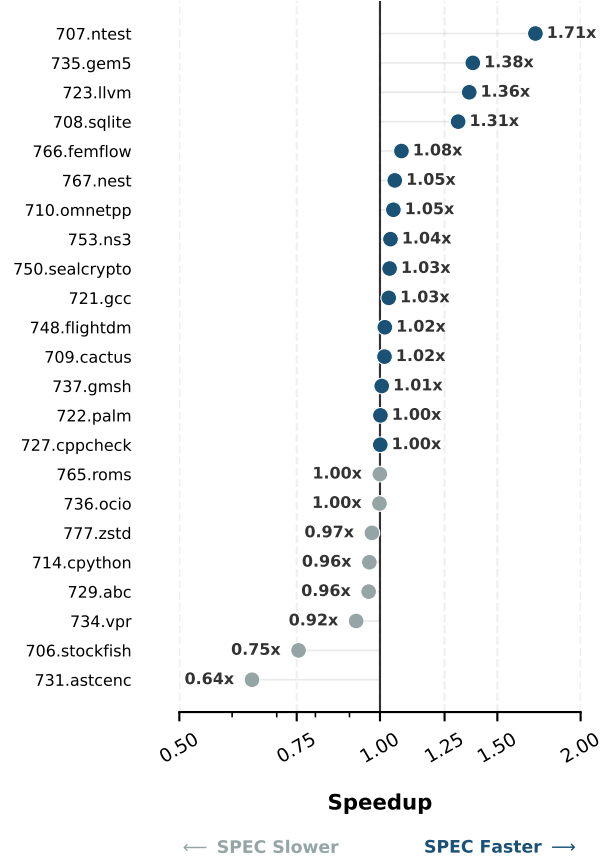


Fig. 2: Fully-loaded performance comparison with 192 copies running on AmpereOne. Values greater than 1.0 indicate speedup relative to the original application.

B. Multi-Copy Results

The multi-copy results, shown in Figure 2, provide the clearest evidence of why these adaptations are critical. While most benchmarks maintained similar trends, the results for several stand out. For 707.ntest, 735.gem5, and 723.llvm, the adapted SPEC benchmark becomes far more efficient than the upstream application. This demonstrates that the I/O operations present in the original code create a severe system-level bottleneck under heavy, multi-tasking load, which would have otherwise obscured the CPU’s true user-level performance. 706.stockfish improved from $0.62\times$ in single-copy to $0.75\times$ in multi-copy, suggesting that the bottlenecks of scaled execution are consistent in both binaries. As for the benchmarks where the SPEC binary outperformed the upstream build, a possible explanation is resource contention: 766.femflow and 767.nest are positive outliers in single-copy but their delta reduces in multi-copy. Factors such as cache pressure, memory bandwidth, and CPU scheduling overhead may contribute to this effect.

C. Other Factors

System time in the upstream applications was compared against total execution time; since SPEC benchmarks target user-level compute performance, system time was intentionally reduced. We attribute 8.1 of the 14 percentage-point delta observed in GCC (single-copy) to this reduction; all other upstream applications showed system time well under 1%. Build tooling can also matter: for Cppcheck, identical `-O3` flags produced different runtimes depending on toolchain and configuration nuances, suggesting some deltas may stem from unscrutinized Makefile differences.

IV. BENCHMARK NOTES

Building each upstream application required benchmark-specific workarounds (e.g., dependency installation, removed command-line flags, or reconciling forked source trees); a full per-benchmark build log is omitted here for space. [Table II](#) summarizes the root cause of every measured delta; below we detail the largest outliers.

706.stockfish. The original Stockfish 15 [18] outperformed the SPEC binary by roughly 38%. Profiling attributes this to `NNUE::evaluate(...)`, whose execution time dropped by 77% in the upstream build due to processor-specific intrinsics that SPEC removed for portability.

707.ntest. The ntest build [13] originally printed and flushed output after every character; SPEC reduced this to one flush per board, a 100× reduction in write syscalls, explaining the large multi-copy speedup. Fundamentally this is a discrepancy in how the community uses the ntest tool; typically a user runs one board at a time, not thousands like in the benchmark, so the I/O issues only manifest at scale.

708.sqlite. For SQLite [7], the dominant runtime delta was isolated to the `-testset fp` workload and is attributable to SPEC changes in `sqlite3.c`. Function-level profiling confirms that the difference is concentrated in software floating-point emulation routines, with the largest deltas in `__multf3` (43% of runtime), `__divtf3` (7.6%), and `__sfp_handle_exceptions` (7.0%). This evidence indicates that increased floating-point conversion/arithmetic overhead drives the performance gap. The benchmark developer confirmed that the `long double` type was changed to `double` for portability and fairness, since `long double` is implementation specific.

723.llvm. For LLVM/Clang [10], the SPEC benchmark adds a `sha512sum` to verify the output in memory, to reduce the I/O of writing to disk. A `sha512sum` on the full text output from the upstream binary matched with the SPEC output, proving equivalence of work. This I/O reduction scales well, leading to a 36% perf increase at high copy count.

731.astcenc. For ASTCENC [2], SPEC’s removal of assembly-language intrinsics for portability accounts for the observed 35% slowdown.

735.gem5. In gem5 [16], the upstream binary performs an I/O dump of large input configuration files on every run. SPEC suppressed this I/O as it was unnecessary for verification,

explaining the upstream binary’s slowdown at high copy counts.

721.gcc. We used Ubuntu’s cross-compiler package `gcc-11-x86-64-linux-gnu` rather than the SPEC redistributable tarball, giving what we believe is the most direct comparison to a GCC- build actually used in the field.

766.femflow. For Femflow [8], SPEC spliced out unnecessary mutexes and locks since this is a single-threaded benchmark. This explains the 18% uplift with single-copy.

767.nest. The NEST simulator [6] uses RNG distributions, which SPEC replaced with ‘`specrand`’. Additionally, `thread_local` variables were removed. These two explain the perf upside of the benchmark at single-copy.

V. CONCLUSION

This study set out to replace the qualitative critique that “benchmarks are not real workloads” [4] with a quantitative analysis of the fidelity gap between SPEC CPU@2026 and its open-source counterparts. Our findings reveal a nuanced reality: for most benchmarks, the adapted SPEC version correlates closely with the original application, especially single-copy, showing that SPEC’s adaptation process often preserves core performance characteristics while achieving portability and determinism.

The most insightful results lie with the outliers. Where SPEC is significantly slower, the cause is typically the removal of non-portable, hand-tuned assembly or ISA intrinsics; where SPEC is significantly faster, the cause is the deliberate reduction of I/O, RNG, and threading bottlenecks. The multi-copy results show these bottlenecks are amplified under parallel load—precisely the conditions under which SPEC’s own warning against cross-comparisons is most justified. In both scenarios, the divergence is a direct consequence of the adaptation process, where the benchmark is modified to isolate the portable algorithms and computational workloads. Therefore, our work does not invalidate SPEC’s warning against direct comparisons; rather, it provides the empirical data that *validates* it. The benchmarks can indeed be different, and our findings illustrate precisely why these principled differences are essential for a tool designed to measure computational throughput.

This work provides a data-driven framework for understanding the inherent trade-offs in creating a scientific benchmark: the fidelity gap is a quantifiable consequence of enforcing portability, determinism, and CPU-bound focus, not evidence of a flaw. By measuring it benchmark-by-benchmark, we give the community the context to interpret SPEC CPU 2026 results correctly, appreciating both its close correlation to real-world applications and the specific, principled reasons for divergence. Furthermore, the methodology presented here offers a template for future benchmark development, providing an empirical basis for committees to make data-driven decisions on when a candidate has diverged too far from its real-world counterpart to remain representative.

REFERENCES

- [1] Ampere, *AmpereOne® Product Brief*, 2024, accessed: May 2, 2026. [Online]. Available: <https://amperecomputing.com/briefs/ampereone-family-product-brief>
- [2] Arm Limited, “Arm® Adaptive Scalable Texture Compression (ASTC) Encoder,” <https://github.com/ARM-software/astc-encoder/tree/2319d9c4>, 2021, gitHub commit 2319d9c4.
- [3] L. Bonebakker, “Comparison of the SPEC CPU Benchmarks with 499 Other Workloads Using Hardware Counters,” in *Proceedings of the SPEC International Workshop on Performance Evaluation: Metrics, Models and Benchmarks*, ser. SIPEW ’08. Berlin, Heidelberg: Springer-Verlag, 2008, p. 144–153. [Online]. Available: https://doi.org/10.1007/978-3-540-69814-2_10
- [4] D. Brown, “AWS re:Invent 2023 - Compute innovation for any application, anywhere,” 2023. [Online]. Available: www.youtube.com/watch?v=dxm93_qgRzk&t=2210s
- [5] X. Chen, L.-H. Hung, R. Cordingley, and W. Lloyd, “X86 vs. ARM64: An Investigation of Factors Influencing Serverless Performance,” in *Proceedings of the 9th International Workshop on Serverless Computing*, ser. WoSC ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 7–12. [Online]. Available: <https://doi.org/10.1145/3631295.3631394>
- [6] J. Hahne *et al.*, “NEST 3.7,” commit: bf55cc4a. [Online]. Available: <https://github.com/nest/nest-simulator.git>
- [7] D. R. Hipp, “SQLite.” [Online]. Available: https://sqlite.org/releaselog/3_33_0.html
- [8] M. Kronbichler, “FemFlow: A benchmark component of the exadg framework,” 2026, commit: 95acc98. [Online]. Available: <https://github.com/kronbichler/spec-femflow>
- [9] R. Li, A. Jacob, N. J. Yadwadkar, and L. K. John, “SPEC CPU2026: Characterization, Representativeness, and Cross-Suite Comparison,” 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2605.03713>
- [10] LLVM Developer Group, *LLVM Compiler Infrastructure, Version 14.0.0*, 2022, software available from GitHub. [Online]. Available: <https://github.com/llvm/llvm-project>
- [11] M. Madhav *et al.*, “SPEC CPU: The Next Generation,” in *Proceedings of the 53rd Annual International Symposium on Computer Architecture*, ser. ISCA ’26. IEEE, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2605.01575>
- [12] R. Panda, S. Song, J. Dean, and L. K. John, “Wait of a Decade: Did SPEC CPU 2017 Broaden the Performance Horizon?” in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 271–282. [Online]. Available: <https://doi.org/10.1109/HPCA.2018.00032>
- [13] V. Petric, “ntest: Othello program,” 2026, SPEC version based on commit: fc7d6b26. [Online]. Available: github.com/vladpetric/ntest
- [14] SPEC, “Step 2 Rules of the CPUv8 Benchmark Search Program,” 2020. [Online]. Available: www.spec.org/cpu/cpuv8/#step2
- [15] —, “SPEC CPU 2026 Docs: Are the benchmarks comparable to other programs?” 2026. [Online]. Available: <https://www.spec.org/cpu2026/Docs/overview.html#Q26>
- [16] The gem5 Development Team, “The gem5 Simulator, Version 22.1.0.0,” 2022. [Online]. Available: <https://www.gem5.org/project/2022/12/30/gem5-22-1.html>
- [17] The Statsig Team, “Real-world vs benchmark performance: Closing the gap,” 2025. [Online]. Available: www.statsig.com/perspectives/realworld-vs-benchmark-performance
- [18] The Stockfish developers, “Stockfish,” 2026. [Online]. Available: <https://github.com/official-stockfish/Stockfish>
- [19] J. v. Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, “How to Build a Benchmark,” in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, ser. ICPE ’15. ACM, 2015, p. 333–336. [Online]. Available: <https://doi.org/10.1145/2668930.2688819>
- [20] Veedrac, “The fallacy of ‘synthetic benchmarks,’” 2020. [Online]. Available: www.reddit.com/r/hardware/comments/jvq3do/the_fallacy_of_synthetic_benchmarks

ACKNOWLEDGEMENTS

Given that each workload had its own unique build process, we used OpenAI Codex 5.4 to help navigate and understand the original sources. We felt like we were retracing the steps the SPEC CPU committee took during CPUv8 Step 2 [14], dealing with the one-time engineering effort of porting workloads. AI proved particularly valuable understanding the build processes, along with assisting to script the runs and plot the results.

We used Google Gemini 2.5 during the editing process of this paper, for making word choices and refactoring our prose into academic writing. All the text was reviewed and revised by the authors.

We thank the benchmark developers for helping us understand the changes that were needed to adapt applications into benchmarks: Amin Mohaghegh (vpr), Vlad Petrić (ntest), Alan Mischenko (abc), Prasad Joshi (marian), Frédérique Silber-Chaussumier (sqlite), and Ronen Zohar (zstd). Gratitude is also extended to the SPEC CPU committee for scrutinizing this work, providing valuable feedback, and pushing us to root-cause the miscorrelations all the way to completion.